DESIGN OPTIONS — FOR DISCUSSION, NOTHING BUILT

Multi-Tenant Job-Seeker Platform: Onboarding & Credential Options

> **Vision stated:** thousands of job seekers share résumés with the platform; the system applies to hundreds of relevant jobs per seeker via AI-customized résumés, across hundreds of job platforms/company career sites.
>
> **Why this needs a real decision, not just more code:** today's codebase is a single-user, local-first tool — one person's résumé, one person's config, on their own machine. What's described above means holding *other real people's* PII, résumés, and — built the most literal way — their job-portal login credentials, at scale. Different risk class (a breach exposes thousands, not one), different legal posture (ToS across "hundreds" of platforms, data-protection obligations), different engineering problem (multi-tenant isolation, horizontal scaling). This document lays out the real options before anything gets built.

1 · The central hard problem: credentials

Everything else here is secondary to one question: **how does the system act on a job platform on a user's behalf without becoming a giant, centralized liability if breached?** Four options, increasing capability and increasing liability:

A No credentials at all — "Tailoring as a Service"

The platform never touches job-portal logins. A user uploads a résumé + facts once; submits job descriptions; gets back a tailored, ATS-safe résumé PDF. **Actual applying stays on the user's own machine/VM** — exactly the self-hosted deployment already built tonight. The SaaS layer is purely the AI-tailoring engine (already built) plus account/résumé management.

Data held: résumés, facts, tailored variants. No passwords, no cookies, no job-portal access.

Liability if breached: real but bounded — nothing lets an attacker act as the user anywhere.

Tradeoff: doesn't deliver "we apply for you" — users still operate their own apply agent. Materially safer, ships fastest.

B Session-cookie delegation (not passwords)

User logs into each platform themselves, then exports the session cookie — this exact mechanism (`auth.method: cookies`) already exists in the codebase today. The platform stores the cookie, never the password. Narrower blast radius (can't reset the account/email on most platforms), self-expires, the user never types a real password into a third party.

Data held: encrypted session cookies per user per platform, decrypted only inside an isolated worker at point of use.

Liability if breached: meaningful — needs a real secrets vault (§2), not a database column.

Delivers: the actual "we apply for you" experience, materially smaller blast radius than raw passwords.

C OAuth delegation where platforms support it

For platforms offering "Sign in with Google" (Wellfound/Uplers already use this), the user authorizes via OAuth; the platform stores a scoped, individually-revocable refresh token — never a password.

Coverage: partial — most of the 7 current platforms are cookie/password-based, not OAuth. Doesn't generalize alone.

Best used: as a supplement to Option B for the subset where it's available.

D **Full custodial automation (stored passwords, centrally executed)**

The platform stores real login credentials (encrypted) and runs automation centrally. Highest capability, matches the vision most literally — and the highest liability: a breach here is a breach of thousands of people's actual passwords, the kind of incident that ends up in security news and regulatory inquiries. Not an "encryption makes it fine" problem — centralized password custody at this scale is a genuinely different risk category.

Typical prerequisites before this is responsible: a security audit / SOC2-style posture, legal review of ToS exposure per platform, explicit informed consent, usually a small invite-only beta first.

Recommendation: don't start here. A later, deliberate decision once A/B validate the product — not the default MVP shape.

2 · Tenant data model — three shapes

Shape 1 — Classic multi-tenant database

One Postgres instance, every table carries a `tenant_id`. Standard SaaS pattern, cheapest per-user, most scalable — but concentrates all tenant data (and any credentials) in one place, the single biggest blast-radius shape.

Shape 2 — Isolated per-tenant deployment

Each job seeker effectively gets their own instance — literally what's built tonight (one `config.yaml`, one `profile/`), orchestrated by a thin management layer. No single database of everyone's data/credentials exists — a breach of one tenant doesn't expose anyone else. Costs more per user; "thousands of isolated VMs" is a real operational problem — but sidesteps the "one big breach" risk by construction.

Shape 3 — Hybrid (the usual real-world answer)

Central Postgres for non-sensitive metadata (accounts, subscription, résumé pointers, history, analytics) — genuinely benefits from being centralized/queryable. Sensitive material (credentials/cookies, if B/C/D is ever adopted) lives in a dedicated secrets vault, isolated from the metadata store, decrypted only inside a short-lived per-tenant context at point of use. The standard shape for "SaaS that must hold some customer secrets."

3 · Recommended phased MVP scope

Each phase independently shippable, adds capability, and only takes on more liability once the previous phase has validated demand:

PHASE	WHAT IT IS	CREDENTIAL MODEL	DATA HELD	SHIPS FROM
0	Ship tonight's tool as self-hosted / open-source	None — user's own machine	Nothing centralized	Already ~built
1	"Résumé Tailoring API" — hosted AI tailoring, self-hosted applying	Option A (none)	Résumés, facts, tailored variants (Shape 1 fine — no secrets)	Reuses `tailoring/` as-is
2	Managed apply automation, invite-only beta	Option B (+C where available), Shape 3	Cookies/tokens in vault, metadata in Postgres	New: worker orchestration, vault integration
3	Broader scale, more platforms, general availability	Same as Phase 2, hardened	Same, at scale	After Phase 2 validates the security posture

Why start at Phase 1, not Phase 2 or the full vision: ships the most differentiated, hardest-to-replicate part of the product (the tailoring engine, already built) with close to zero new liability — a real, monetizable, testable product on its own. It also naturally produces the usage data (tailoring request volume, JD pattern clustering, local-LLM compute per request) that makes sizing Phase 2's infrastructure a real estimate instead of a guess.

4 · Open questions — yours to decide, not answerable by architecture alone

1. **Is "we apply on your behalf" (B/C/D) actually the product**, or would "AI tailors your résumé, you still click apply" (A) satisfy the core value proposition for a first version? The single highest-leverage scope decision.
2. **If B/C/D:** what's the acceptable legal exposure across "hundreds" of platforms' ToS? Worth a real legal read before committing engineering time, not after.
3. **Target user:** "help me personally + close friends/family" (small n, lower stakes, could reasonably start at Shape 2 / Option B directly) or genuinely "thousands of strangers" (needs the fuller Phase 0→3 sequencing and a real security/legal posture first)?
4. **Monetization model** (if any) — affects infra sizing and whether Phase 1 alone is a viable standalone product or just a stepping stone.

> Nothing above is implemented. Whenever you want to move on any of it, the natural next step is picking a phase and an option from §1/§2 — at which point this becomes a real SDA document with diagrams, following the same process as the résumé-tailoring feature.

docs/design/multi-tenant-onboarding-options.md · jobs-auto-apply-resume-customize